

awesome-mcp-servers 小白指南

GitHub 上最全的 MCP (Model Context Protocol, 模型上下文协议) 服务器精选清单 —— 给 AI 装上"手和脚"的工具大全。本指南按"工科毕业生第一次接触"的 12 个疑问层层展开, 最后有一个亲手验证过的动手实验。

punkpeye/awesome-mcp-servers

★ GitHub 热门清单

含 4086 个服务器条目

约 60 个领域分类

4086

收录的 MCP 服务器

~60

领域分类目录

8

种编程语言实现

1

个协议: MCP

Q1

MCP 到底是什么? 一句话能说清吗?

MCP (Model Context Protocol, 模型上下文协议) 是 AI 应用的"USB-C 接口"——一个开放标准, 让任何大模型应用 (Claude Desktop、Cursor、Claude Code.....) 都能用同一种方式连接外部工具和数据源。



图1: 不管左边是谁、右边是谁, 中间都用同一个 MCP 标准对话——就像所有设备都改用 USB-C 充电口

它由 Anthropic 在 2024 年 11 月发布并开源, 现在已经是 AI 工具生态的事实标准。这份清单收录的每一个"MCP 服务器", 就是一根"插在 USB-C 上的设备": 有的让 AI 读文件, 有的让 AI 查数据库, 有的让 AI 操作浏览器。

Q2

为什么需要它？没有 MCP 之前人们怎么做？

早期让 AI 调用外部工具，靠的是各家自己发明的 "Function Calling"（函数调用）格式。每接一个新工具，都要为每一个 AI 应用单独写一遍对接代码。

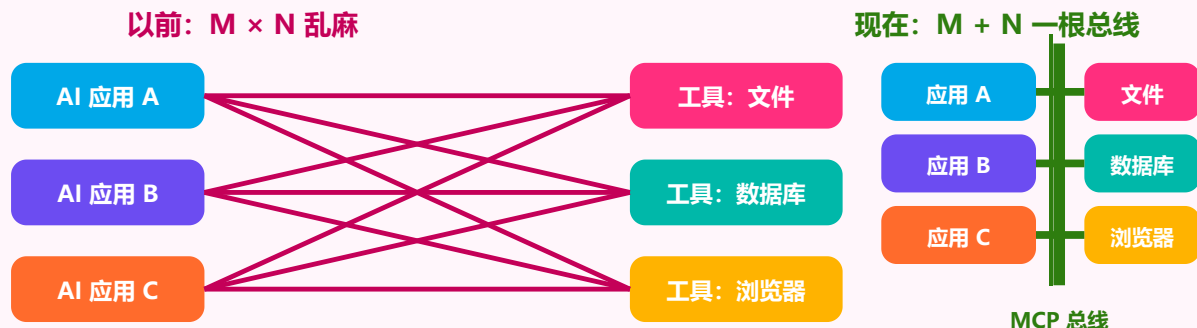


图2: 3 个应用 $\times$ 3 个工具, 以前要写 9 套对接; 有了 MCP, 各写一次即可 ($3+3$)

MCP 把 " $M \times N$ " 的对接爆炸变成 " $M + N$ ": 应用方只需实现一次 MCP 客户端, 工具方只需实现一次 MCP 服务器, 双方即插即用。这正是所有"协议" (HTTP、USB、蓝牙) 的取胜逻辑——先统一语言, 再谈连接。

Q3

MCP 的整体架构是什么样的？

MCP 是经典的"客户-服务器"架构，但有**三个角色**，容易混淆：

角色	是什么	例子
Host (宿主)	你直接使用的 AI 应用，负责跑大模型	Claude Desktop、Cursor、Claude Code
MCP Client (客户端)	宿主内部维护的连接器， 一个连接对应一个 ，负责和某个服务器说话	宿主程序里内置的 MCP 模块
MCP Server (服务器)	独立的工具提供方，把某种能力（文件/数据库/浏览器...）包装成标准工具	本清单里的 4086 个项目

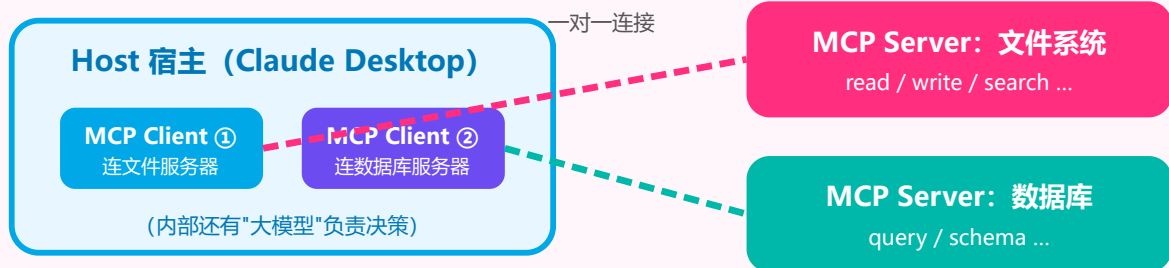


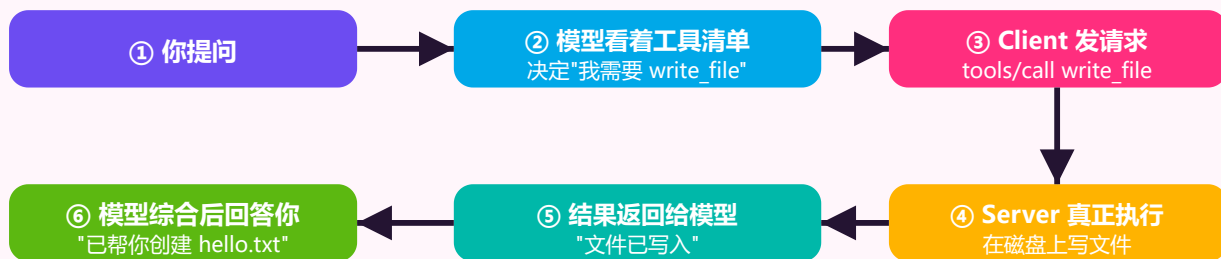
图3：一个 Host 里可以同时插多个 Client，每个 Client 只和一台 Server 通话

类比：Host 是**电脑**，Client 是**USB 口**，Server 是**U 盘/键盘/摄像头**。电脑不会直接跟 U 盘里的闪存芯片打交道，中间永远隔着标准化的接口层。

Q4

一次对话里，工具调用到底是怎么发生的？

关键认知：**大模型本身“不会”执行任何工具——它只会“点名”**。真正动手的是 MCP 服务器。完整循环如下：



开始前还有一步：Client 先用 `tools/list` 问 Server “你有哪些工具”，把清单塞进模型的上下文

模型 = 点菜的顾客 | Client = 服务员 | Server = 后厨

图4：工具调用的完整生命周期——模型决策、Client 传话、Server 干活

注意第②步：模型选工具的依据是**工具的名字和描述写得好不好**。这也是清单里有“TDQS（工具描述质量评分）”教程的原因——描述含糊的工具，模型根本想不起来用。

Q5

MCP 服务器本质上是什么？消息长什么样？

MCP 服务器本质就是一个“会说 JSON-RPC 的普通程序”——没有魔法。最常见的形式是命令行进程：宿主把它作为子进程启动，通过标准输入/输出（stdio）一行一行地互发 JSON 消息（也有走 HTTP+SSE 的远程形式）。

MCP 建立在 **JSON-RPC 2.0** 之上，日常只用到三种消息：**请求**（要结果）、**响应**（给结果）、**通知**（不用回复）。实际报文：

```
{
  "jsonrpc": "2.0", "id": 1, "method": "initialize",
  "params": { "protocolVersion": "2024-11-05",
    "clientInfo": { "name": "demo", "version": "0.1" } }
}
→ 服务器回： {"result": {"serverInfo": {"name": "filesystem", "version": "0.2.0"}, ...}}
```

协议约定了几个核心方法，全部一眼能懂：

方法	方向	作用
<code>initialize</code>	Client → Server	握手：报版本、报身份（每次连接必做）
<code>tools/list</code>	Client → Server	"你有哪些工具？"——返回名字+描述+参数表
<code>tools/call</code>	Client → Server	调用某个工具，附带参数
<code>notifications/initialized</code>	Client → Server	通知：握手完成（无需回复）

正因为如此简单，**实验里我们甚至不需要任何 AI，就能直接和服务器“聊”起来**——后面的动手实验会亲手证明这一点。

Q6

这份清单长什么样？4086 个条目怎么逛？

它就是一个超大 README.md，结构固定，五分钟能摸清：



它还有姊妹清单: [awesome-mcp-clients](#) (客户端)、[awesome-remote-mcp-servers](#) (免安装、连 URL 就用的托管服务器)、[awesome-mcp-devtools](#) (开发调试工具)。本清单专注于"你自己 clone 下来跑"的服务器。

Q7

服务器有哪些典型门类？各解决什么问题？

约 60 个分类可以归成六大脑洞方向，每类挑一个代表感受一下：

方向	代表分类	典型能力
让 AI 碰你的数据	File Systems、Databases、Knowledge & Memory	读写本地文件、查 Postgres/SQLite、给 AI 加长期记忆（含官方 Obsidian 读取工具 seekstone）
让 AI 碰你的电脑	OS Automation、Command Line、Browser Automation	操作键盘鼠标、执行 shell 命令、开 Chrome 自动点网页
让 AI 碰互联网	Search、Social Media、E-Commerce	联网搜索、发推/发帖、查商品价格
让 AI 碰开发流程	Version Control、Coding Agents、Developer Tools	操作 git/GitHub、派子任务给编程 agent
让 AI 碰云服务	Cloud Platforms、Communication、Finance	管 AWS/阿里云、发 Slack 消息、查股票
脑洞类	Gaming、Home Automation、Sports、Spirituality	玩游戏、开智能灯泡、查赛程.....

💡 看懂门类 = 看懂整个生态的野心：凡是"有 API 或有界面"的东西，都在被包成 MCP 服务器。以后遇到"AI 能不能操作 X"的问题，先来这查 X 分类。

Q8

条目前的 🏆 🐍 🏠 ☁️ 是什么意思？怎么挑一个靠谱的？

清单定义了一套图例（Legend），每个条目前的 emoji 是它的"身份证"：

维度	符号 → 含义
权威度	🏆 = 官方实现（如 Anthropic、Aiven 官方出品，优先选）
语言	🐍 Python 📄 TypeScript/JS 🦄 Go 🦀 Rust # C# ☕ Java 🐼 C/C++ 💎 Ruby
运行位置	🏠 本地服务（操作本机软件，如控制 Chrome） ☁️ 云服务（调远程 API，如天气） 🖨️ 嵌入式
操作系统	🍏 macOS 🪟 Windows 🐧 Linux

挑选漏斗：先 🏆 官方 → 再 🪟 确认支持 Windows → 再选顺手的语言 🐍/📄 → 最后看条目里的 glama.ai 评分徽章（机器评的工具描述质量分）。清单头部还有 TDQS 教程解释这套评分——描述越清晰的工具，模型用得越准。

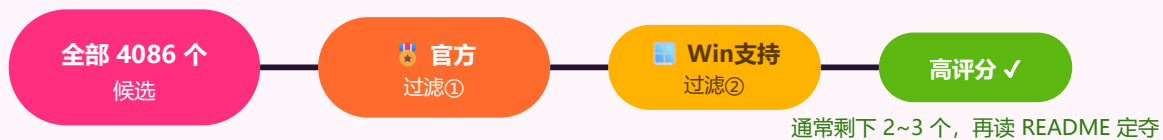


图6：三步漏斗，从 4086 缩到个位数候选

Q9

MCP 和 Function Calling、LangChain 工具是什么关系？

三个概念经常被混着说，其实是**不同层次**的东西：



发动机 ← 标准传动轴 → 整车厂：MCP 不取代前两者，而是夹在中间当“通用传动轴”

图7：三层各司其职——能力层、标准层、应用层

模型最终“动手”靠的还是 Function Calling 这类原生能力；MCP 只是让“工具的供货方式”标准化了。框架（LangChain、CrewAI）也纷纷加了对 MCP 的适配——一个 MCP 服务器写好后，可以被所有支持它的框架直接挂载，这正是协议的价值。类比知识库里已有的概念：MCP 对工具生态的意义，类似 [[构建流水线与 Pass]] 里“统一接口的 Pass”——每个阶段按约定接入，流水线即可自由替换组件。

Q10

我想自己写一个 MCP 服务器，难吗？

不难——这是 MCP 生态最惊喜的地方。**用 Python 的 FastMCP 框架，一个普通函数加一行装饰器就是一台 MCP 服务器：**

```

from fastmcp import FastMCP

mcp = FastMCP("我的工具箱")

@mcp.tool
def add(a: int, b: int) -> int:
    """把两个整数相加""" # ← 这行注释就是给模型看的"工具描述"!
    return a + b

mcp.run() # 以 stdio 方式启动，等宿主来连
    
```

要点：函数签名 + 类型标注自动生成参数表（JSON Schema）；**docstring 就是工具描述**，直接决定模型会不会用它（呼应 Q4 的 TDQS）。清单的 Frameworks 区还列了 TypeScript 版 FastMCP、Rust 的 TurboMCP、Go 的 gomcp 等——你熟悉哪门语言都有对应 SDK。

🔥 实战路径：把你已有的任何脚本/内部 API 包一层 `@mcp.tool`，它立刻可以被 Claude Desktop / Cursor / Claude Code 调用——这是“给自己的工作流造工具”的最短路径。

Q11

让 AI 读写我的文件、数据库.....安全吗？

两大真实风险，装任何第三方服务器前都要想：

风险	是什么	怎么防
过度授权	服务器拿到的权限大于实际需要（如整个 C 盘）	启动时只授予最小目录/只读模式（实验里我们只给了一个 sandbox 文件夹）
提示注入	恶意网页/文件内容里藏着"指令"，诱骗模型去调用危险工具	敏感操作要人工确认；社区有专门的安全网关类服务器（如清单里的 warden）做工具扫描

类比知识库的 [[质检Gate与自我纠错循环]]：**把每个 MCP 工具调用当成流水线上的一个 Pass，在"AI 想调用"和"真正执行"之间设一道人工/规则 Gate**——宿主（如 Claude Code）默认就会对敏感操作弹出确认，别图省事一路放行。



Q12

本地服务器和远程服务器有什么区别？

本清单收录的都是**本地自装型**；还有一类“远程 MCP 服务器”（另一个姊妹清单）只需在客户端填一个 URL。对比：

🏠 本地（本清单）

☁️ 远程（姊妹清单）

安装	clone/npm 安装，写进宿主配置文件，宿主作为子进程拉起（stdio 传输）	零安装，填 URL 即用（HTTP/SSE 传输）
数据去哪	不出你的电脑，适合隐私敏感场景（本地文件、内网数据库）	数据经过对方服务器，适合 SaaS 类能力
典型例子	filesystem、sqlite、playwright 浏览器控制	各家官网提供的在线搜索/地图/CRM 服务器
运维	自己升级、管依赖	对方维护，但你受制于对方可用性

💡 经验法则：碰“你自己的数据”用本地，碰“互联网上的服务”用远程——和图例里 🏠 / ☁️ 的划分完全一致。

🎁 这个项目给我什么：6 个立刻能用的场景

① 给 AI 助手装“手脚”

把 filesystem / sqlite 服务器接进 Claude Desktop 或 Claude Code，让 AI 直接帮你整理文件、查本地数据库。

② 按需“点菜”不造轮子

要做 AI 应用时先来清单搜一圈：浏览器控制（playwright）、网页抓取、Slack 通知……大概率已有现成服务器。

③ 把自己的脚本变成 AI 工具

用 FastMCP 把写好的 Python 脚本包成 MCP 服务器，从此 Claude Code / Cursor 都能调用它。

④ 给 AI 加长期记忆

Knowledge & Memory 类服务器（甚至有直接读 Obsidian 库的 seekstone）让 AI 跨会话记住你的项目上下文。

⑤ 浏览器/桌面自动化

让 AI 替你操作 Chrome 填表、抓取需要登录的页面数据——Browser Automation / OS Automation 类。

⑥ 了解 AI 能力边界

60 个分类就是一份“AI agent 能干什么”的地图，做技术选型、写方案时直接引用。



动手实验：不接 AI，亲手和 MCP 服务器对话（已全程验证



)

目标：亲眼看到 MCP 协议的真实报文——启动官方文件系统服务器，完成握手、列出工具、调用工具写文件，最后到磁盘上验证。全程不需要任何大模型 API key。

第 1 步：确认环境（1 分钟）

怎么操作：打开 PowerShell，执行：

```
node --version
```

目的：确认 Node.js 已装（MCP 服务器以 npm 包形式分发）。实现了什么：拿到版本号。怎么验证：

✅ 预期结果：输出类似 v24.21.0（v18 以上均可）。若提示找不到命令，先去 nodejs.org 安装 LTS 版。

第 2 步：跑通握手脚本

怎么操作：进入 exercise\ 目录，执行（首次会自动下载服务器包，需等十几秒）：

```
cd F:\reminder\mcp_servers\exercise
python mcp_handshake.py
```

目的：让脚本替我们扮演“宿主”，对服务器发出 initialize 握手。实现了什么：一个真实的 MCP 服务器进程被拉起，双方完成协议握手。怎么验证：

✅ 预期结果：输出 [2] 握手成功！服务器：secure-file-system-server v0.2.0（协议版本 2024-11-05）——你刚见证了 MCP 的“TCP 三次握手时刻”。

第 3 步：看它到底提供哪些工具

怎么操作：无需操作，脚本紧接着自动发送了 tools/list。目的：体验“工具发现”——宿主就是靠这一步知道模型能用的工具清单的。实现了什么：拿到 14 个工具的名字与描述。怎么验证：

✅ 预期结果：输出 [3] 该服务器提供 14 个工具：，随后列出 read_text_file、write_file、edit_file、directory_tree、search_files 等——每个名字都会（在真实宿主里）成为模型可点名的工具。

第 4 步：调用 write_file 工具写一个文件

怎么操作：同样自动执行——脚本发送 tools/call，参数 name=write_file，内容为一句中文。目的：完整走一遍“调用-执行-返回”闭环。实现了什么：服务器真的在磁盘上创建了文件。怎么验证：

✅ 预期结果：输出 [4] tools/call(write_file) 返回 isError=False，以及 [5] 磁盘验证：... \sandbox\hello.txt 存在，内容 = 'Hello from MCP! 这行字是 MCP 工具写的。'，最后打印全部通过 ✅。再亲手打开 exercise\sandbox\hello.txt 看一眼，文件确实在。

第 5 步：改一行代码，感受"授权边界"

怎么操作：用编辑器打开 `mcp_handshake.py`，把启动参数里的 `SANDBOX` 改成你用户主目录（如 `C:\Users\Windows`）外的随意路径或把 `tools/call` 的目标路径改成 `sandbox` 之外的文件，再跑一次：

```
python mcp_handshake.py
```

目的：验证 Q11 讲的"最小授权"不是空话——服务器会拒绝越界操作。**实现了什么：**观察一次失败的工具调用。**怎么验证：**

✓ **预期结果：**越界写入时返回 `isError=True`，错误信息提示路径不被允许（`Access denied / not within allowed directories`）。失败也是实验——你现在亲眼确认了授权围栏真的存在。

实验小结 & 下一步

你已经完成了宿主连接 MCP 服务器的**全部核心动作**：握手 → 发现工具 → 调用工具。真实宿主（Claude Desktop 等）只是在这之上加了"让大模型来决定调哪个工具"。下一步延伸：在 Claude Desktop 的配置文件 `claude_desktop_config.json` 里加上这个同样的 `npx @modelcontextprotocol/server-filesystem` 命令（指向你的某个工作目录），就能用自然语言让 AI 操作文件——你会发现它说的每一句话，背后都是你今天亲手发过的那种 JSON 消息。



坑与备忘

■ **中文 Windows + Python：**跑脚本前建议 `setx PYTHONUTF8 1`，避免 GBK 编码导致中文读写报错（本实验脚本已显式用 UTF-8，不受影响）。

■ **npx 首次运行慢：**第一次 `npx -y @modelcontextprotocol/server-filesystem` 会下载包，别以为是卡死；之后再跑是秒起。

■ **stdio 服务器不"打印"：**MCP 服务器的 `stdout` 是协议通道，调试信息走 `stderr`——写自己的服务器时往 `stdout` 随便 `print` 会直接破坏协议。

■ **协议版本：**握手时报文里的 `protocolVersion` 需要双方都认识（实验用 2024-11-05），版本不匹配会被服务器拒绝。

📖 **学习产出：**本 PDF + `exercise\mcp_handshake.py`（可复跑的握手实验）+ 知识库新增概念：[[MCP模型上下文协议]]、[[JSON-RPC与stdio传输]]、[[工具调用生命周期]]、[[M×N集成问题]] | 仓库：github.com/punkpeye/awesome-mcp-servers | 学习日期：2026-10-01